

Titanic Dataset

Capstone Project

Data Source: Kaggle | Tool: Google Colab | Language: Python

Libraries: Pandas • NumPy • Matplotlib • Seaborn • Scikit-learn

About the Dataset

Data Source

Platform: Kaggle (kaggle.com)

Dataset: Titanic: Machine Learning from Disaster

Format: CSV File (Titanic.csv)

Total Rows: 891 passengers

Total Columns: 12 features

Key Columns

PassengerId • Survived • Pclass • Name
Sex • Age • SibSp • Parch • Ticket
Fare • Cabin • Embarked

What We Analyze

- Missing values & data quality
- Outlier detection & treatment
- Correlation between features
- Survival by Gender, Class & Port
- Age & Fare distributions
- Family size impact on survival
- Logistic Regression model training
- Model accuracy & evaluation

Analysis Pipeline

1

Data

2

Analysis

3

Visualization

4

Insights

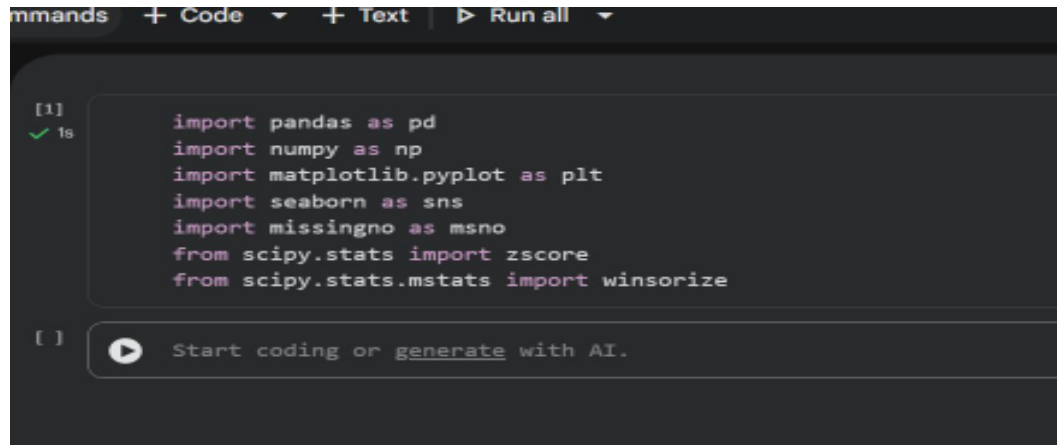
5

Recommendation

Final Goal

Predict whether a passenger survived the Titanic disaster using Logistic Regression, achieving ~79% accuracy.
Analyze key survival factors: Gender, Passenger Class, Age, Fare, and Embarkation Port.

Step 1: Setup & Library Imports



The screenshot shows a Jupyter Notebook interface. At the top, there are tabs for 'Commands', '+ Code', '+ Text', and a 'Run all' button. Below the tabs, there are two code cells. The first cell, labeled '[1]' with a green checkmark and '1s' execution time, contains the following Python code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import missingno as msno
from scipy.stats import zscore
from scipy.stats.mstats import winsorize
```

 The second cell, labeled '[]', contains a play button icon and the text 'Start coding or generate with AI.'

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import missingno as msno
from scipy.stats import zscore
from scipy.stats.mstats import winsorize
```

Libraries Used

pandas: Data manipulation & analysis

numpy: Numerical computations

matplotlib: Base plotting library

seaborn: Statistical visualizations

missingno: Missing value visualization

scipy.stats: Z-score for outlier detection

sklearn: Machine learning models

Step 2: Data Loading & First Look

```
df = pd.read_csv('Titanic.csv')
df.shape
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0

```
print("Column Names:", list(df.columns))
print("Total Columns:", len(df.columns))
print("Total Rows:", len(df))
```

```
Column Names: ['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked']
Total Columns: 12
Total Rows: 891
```



Start coding or [generate](#) with AI.



Step 3: Dataset Structure & Statistics

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

df.info() — Data types & null counts

```
df.describe()
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

df.describe() — Statistical summary

Step 4: Missing Values Detection

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

+ Code

+ Text

```
missing_percent = (df.isnull().sum() / len(df)) * 100
print(missing_percent.round(2))
```

PassengerId	0.00
Survived	0.00
Pclass	0.00
Name	0.00
Sex	0.00
Age	19.87
SibSp	0.00
Parch	0.00
Ticket	0.00
Fare	0.00
Cabin	77.10
Embarked	0.22

dtype: float64

Start coding or generate with AI.

Missing Summary

Age

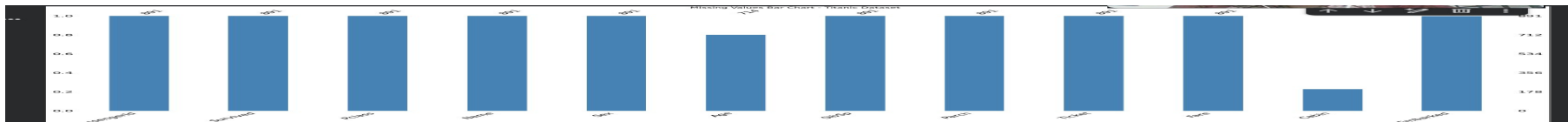
177 rows
19.87%

Cabin

687 rows
77.10%

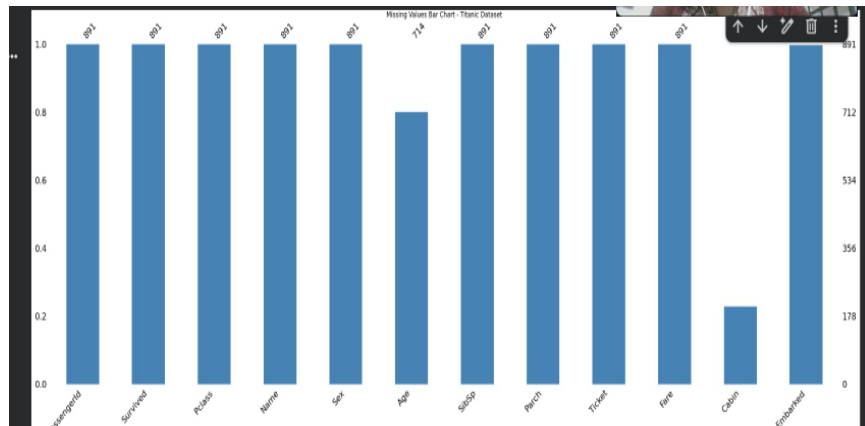
Embarked

2 rows
0.22%

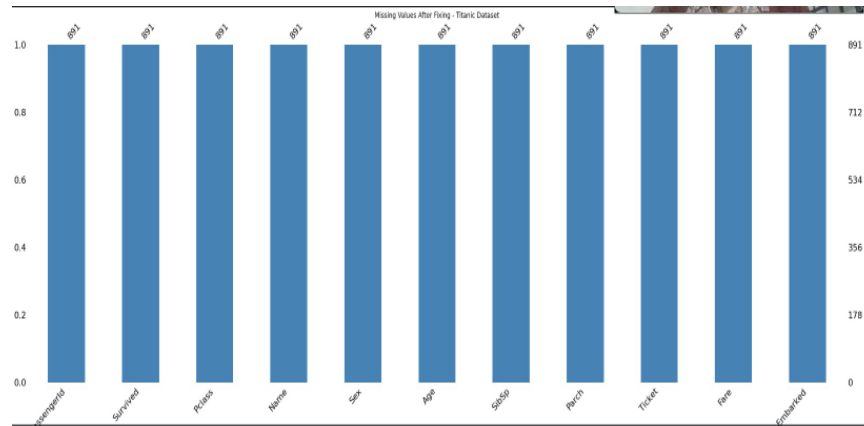


Step 4b: Missing Values – Before & After

Before Fixing:



After Fixing:



Fix Applied:

```
df['Age'].fillna(df['Age'].median(), inplace=True)
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
df.drop(columns=['Cabin'], inplace=True)
print("Missing values after fixing:")
print(df.isnull().sum())
print("\nNew Shape:", df.shape)

Missing values after fixing:
PassengerId    0
Survived        0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Embarked       0
dtype: int64

New Shape: (891, 11)
```

Step 5: Duplicates Check & Data Types

Duplicate Records:

```
df.duplicated().sum()  
np.int64(0)
```

No duplicate records found in the dataset!

Explanation:

- int64 — Integer columns (Survived, Pclass, SibSp, Parch)
- float64 — Decimal columns (Age, Fare)
- object — Text/categorical columns (Name, Sex, Ticket, Embarked)

Data Types (dtypes):

```
df.dtypes
```

```
0  
PassengerId    int64  
Survived       int64  
Pclass         int64  
Name           object  
Sex            object  
Age            float64  
SibSp          int64  
Parch          int64  
Ticket         object  
Fare           float64  
Embarked       object
```

Step 6: Outlier Detection (Z-Score & Boxplot)

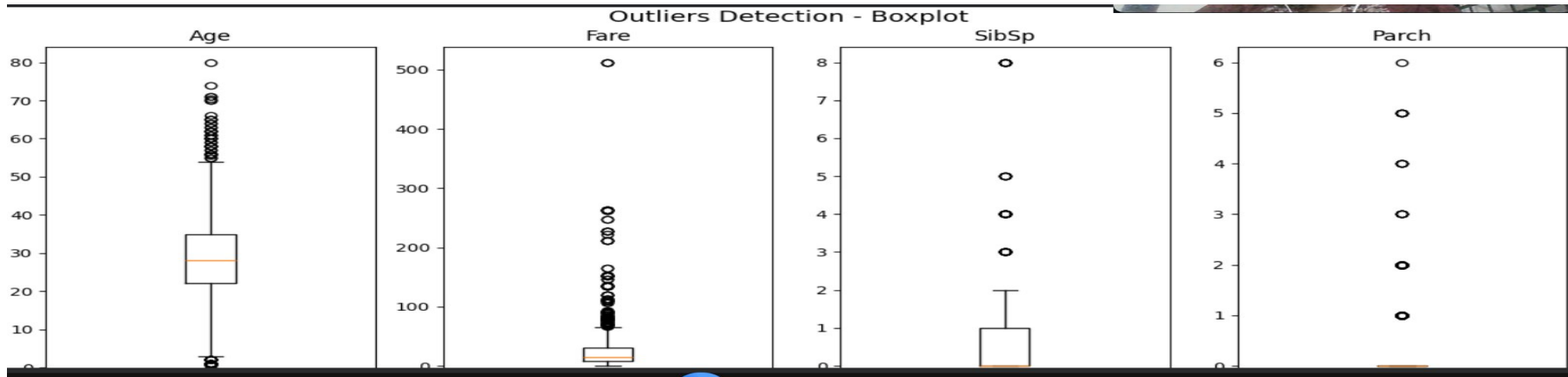
```
numeric_cols = ['Age', 'Fare', 'SibSp', 'Parch']
z_scores = df[numeric_cols].apply(zscore)
outliers = (z_scores.abs() > 3)
print("Outliers per column:")
print(outliers.sum())
```

```
Outliers per column:
Age      7
Fare     20
SibSp    30
Parch    15
dtype: int64
```

Outliers Found (Z-Score > 3):

Age	7 outliers
Fare	20 outliers
SibSp	30 outliers
Parch	15 outliers

Boxplot – Before Winsorization:

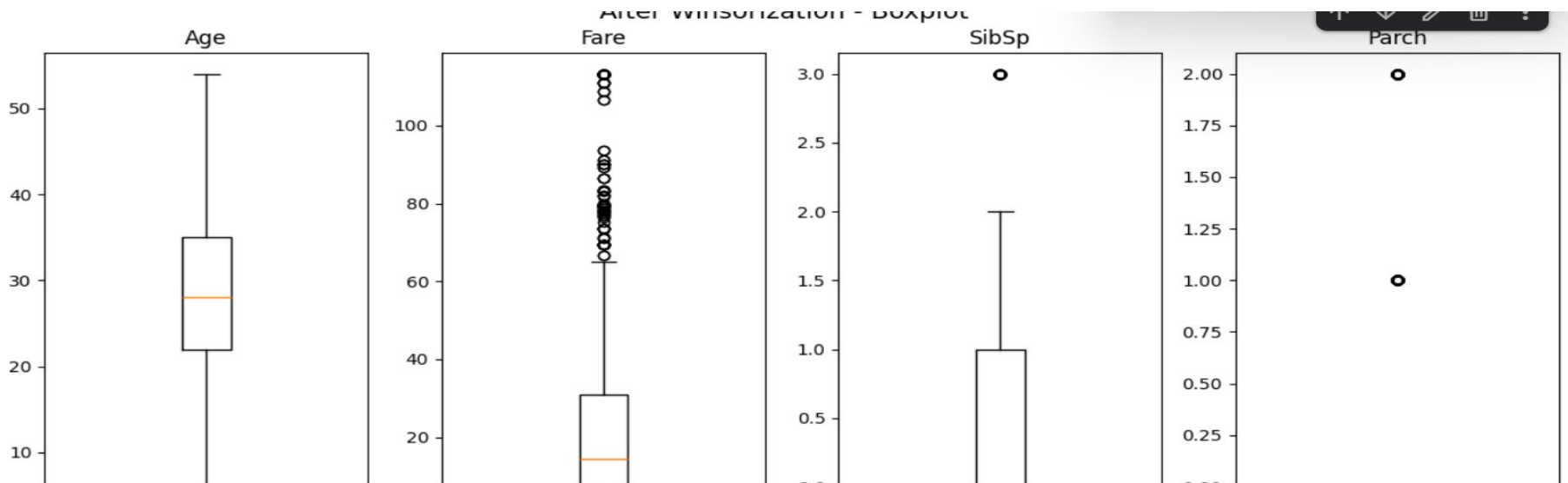


Step 7: Outlier Treatment — Winsorization

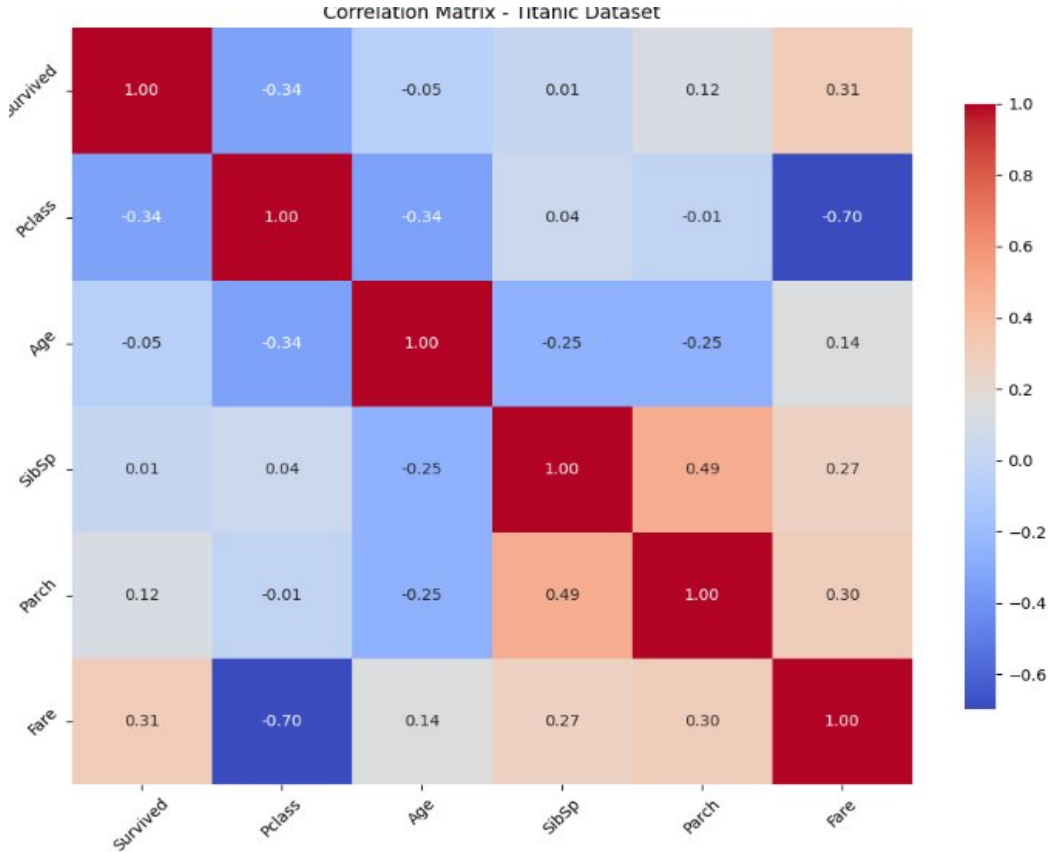
```
[18]
✓ Os
df['Age'] = winsorize(df['Age'], limits=[0.05, 0.05])
df['Fare'] = winsorize(df['Fare'], limits=[0.05, 0.05])
df['SibSp'] = winsorize(df['SibSp'], limits=[0.05, 0.05])
df['Parch'] = winsorize(df['Parch'], limits=[0.05, 0.05])
```

Winsorization caps extreme values at the 5th and 95th percentile — reducing outlier impact without removing data.

Boxplot — After Winsorization:



Step 8: Correlation Matrix



Key Correlations

- Survived ! Pclass: -0.34 (lower class less survival)
- Survived ! Fare: +0.31 (higher fare better survival)
- Pclass ! Fare: -0.70 (strong negative — 1st class pays more)
- SibSp ! Parch: +0.49 (family members travel together)
- Age ! Pclass: -0.34 (older passengers in higher classes)
- Survived ! Age: -0.05 (very weak correlation)

Step 9: Problem Type Identification

```
target_col = 'Survived'
unique_values = df[target_col].unique()
print(f"Target column: {target_col}")
print(f"Unique values: {unique_values}")
print(f"Values: {df[target_col].unique()}")
if unique_values <= 10:
    print("\nProblem Type: CLASSIFICATION")
else:
    print("\nProblem Type: REGRESSION")
```

```
• Target column: Survived
  Unique values: 2
  Values: [0 1]

  Problem Type: CLASSIFICATION
```

Classification Problem

Target Column: Survived

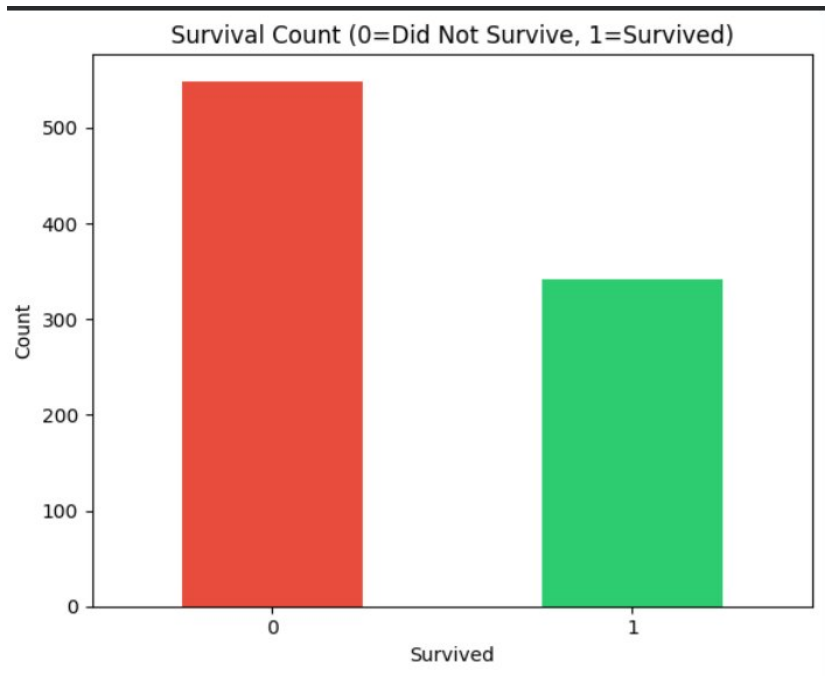
Values: 0 (Did Not Survive)
1 (Survived)

Unique Values: 2

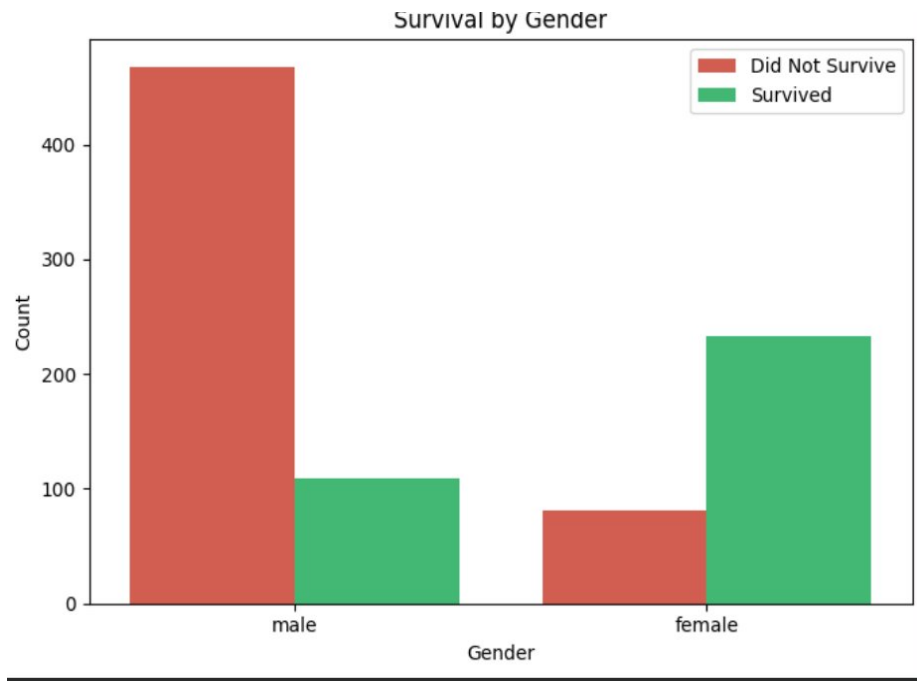
Since target has ≤ 10 unique values
Binary Classification Problem

Step 10: Survival Analysis – Count & Gender

Overall Survival Count:



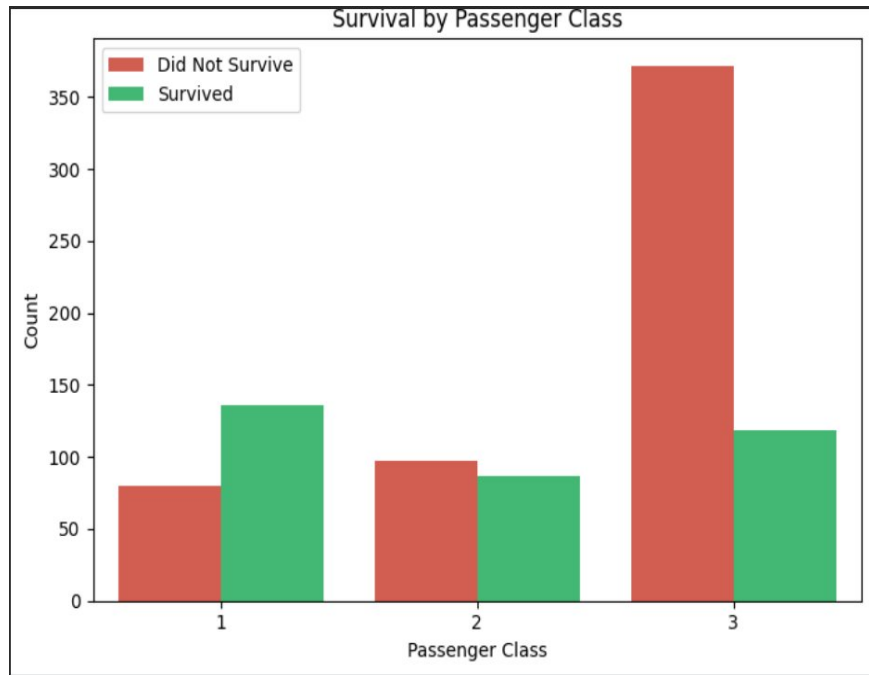
Survival by Gender:



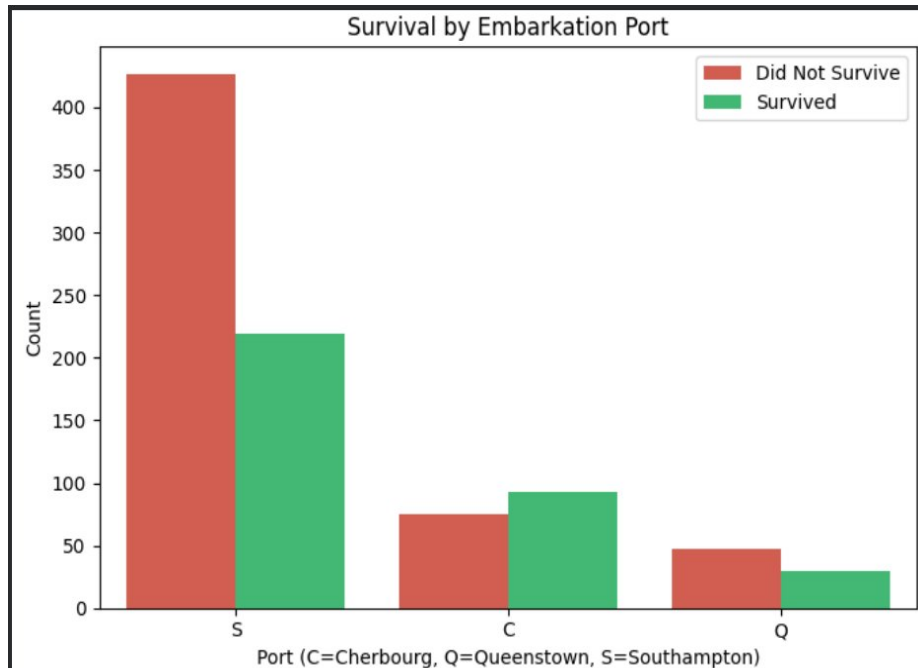
549 did not survive (61.6%) vs 342 survived (38.4%) | Females had significantly higher survival rate than males

Step 10b: Survival by Class & Embarkation Port

By Passenger Class:



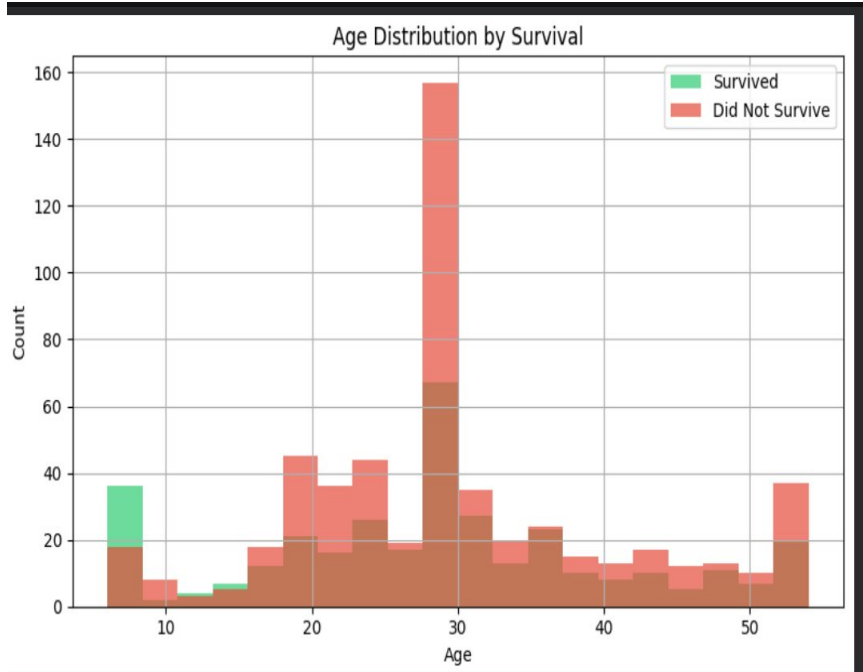
By Embarkation Port:



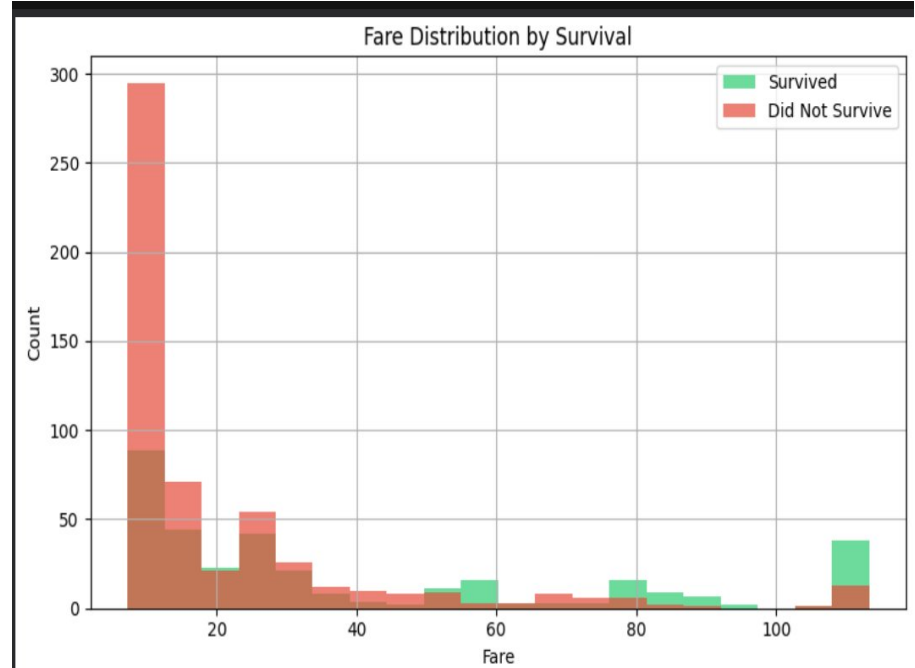
1st class had highest survival | Cherbourg (C) passengers had best survival rate among ports

Step 11: Age & Fare Distribution by Survival

Age Distribution:



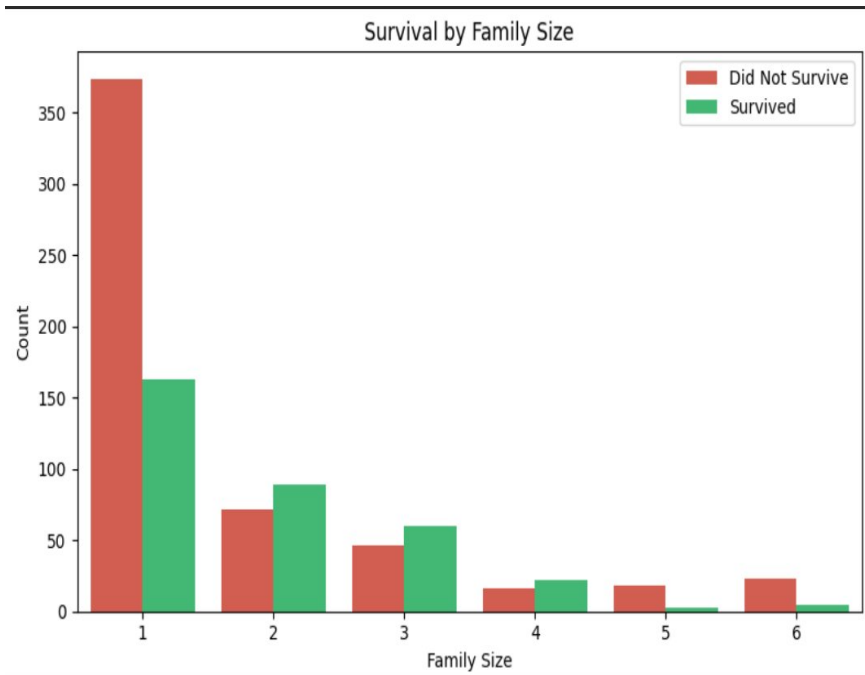
Fare Distribution:



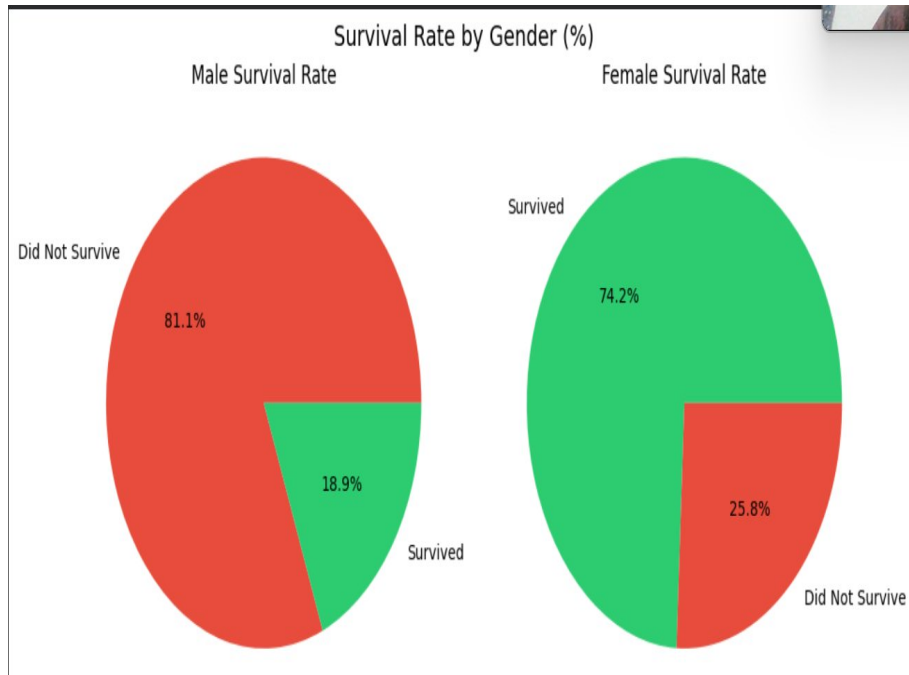
Children had higher survival rate | Passengers paying higher fares (1st class) survived more often

Step 12: Family Size & Gender Survival Rate

Survival by Family Size:



Gender Survival Rate (%):



Solo travelers had lowest survival rate | Female survival: 74.2% vs Male survival: only 18.9%

Step 13: Feature Encoding & Train/Test Split

```
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
df['Embarked'] = df['Embarked'].map({'S': 0, 'C': 1, 'Q': 2})

# Features aur Target select karo
X = df[['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']]
y = df['Survived']

# Train Test Split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)

Training set size: (712, 7)
Testing set size: (179, 7)
```

Encoding Done:

Sex: male 0, female 1

Embarked: S 0, C 1, Q 2

Features (X): Pclass, Sex, Age, SibSp, Parch, Fare, Embarked

Target (y): Survived

Train/Test Split (80/20):

Training set: 712 samples (80%) | Testing set: 179 samples (20%) | random_state = 42

Step 14: Model Training – Logistic Regression

```
# 16. Model Training
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)

print("Model trained successfully!")

# Predictions
y_pred = model.predict(X_test)
print("Predictions done!")

Model trained successfully!
Predictions done!
```

Why Logistic Regression?

Binary classification (0/1)

Works well with mixed features

Interpretable coefficients

Fast training & prediction

```
# 17. Evaluation Metrics
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy*100:.2f}%")

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred,
    target_names=['Did Not Survive', 'Survived']))

Model Accuracy: 79.33%

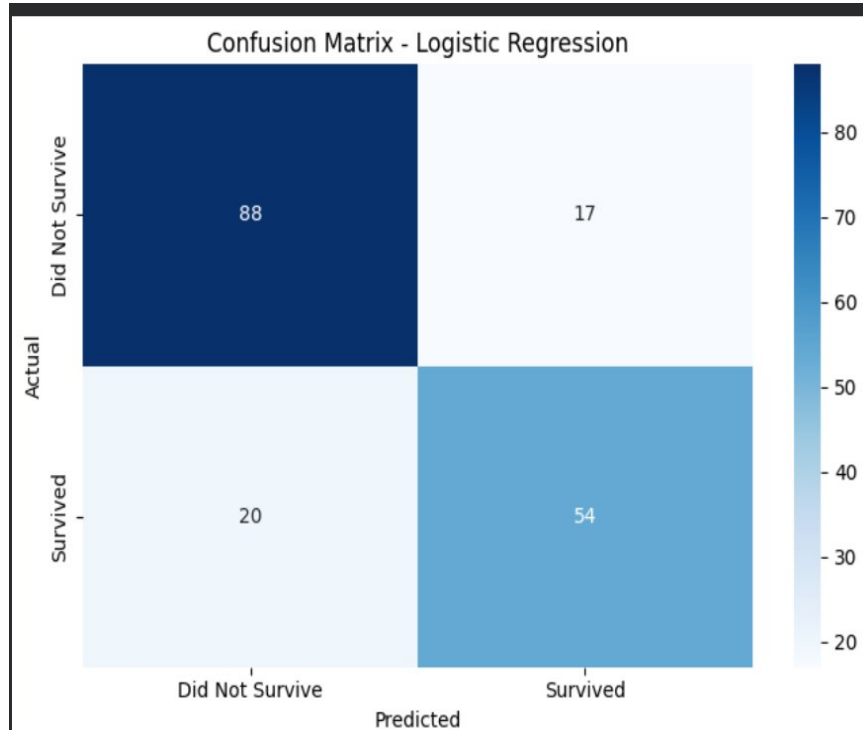
Classification Report:
              precision    recall  f1-score   support

Did Not Survive      0.81      0.84      0.83       105
Survived             0.76      0.73      0.74        74

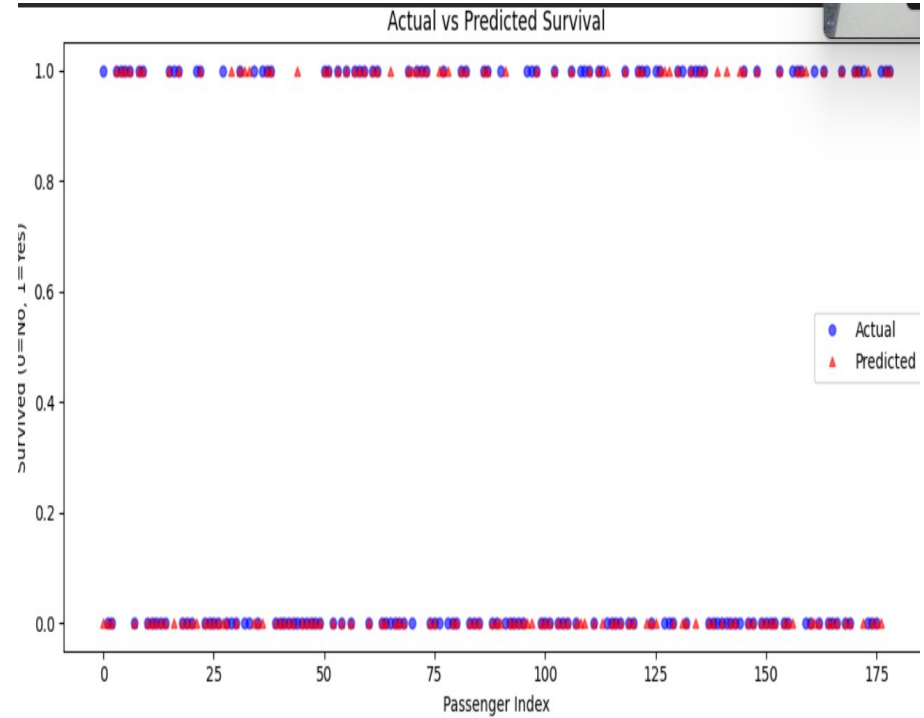
 accuracy      0.79
 macro avg     0.79
weighted avg     0.79
```

Step 15: Confusion Matrix & Predictions

Confusion Matrix:



Actual vs Predicted:



Step 16: Model Accuracy & Saving

```
# 19. Save Model
import joblib
joblib.dump(model, 'titanic_model.joblib')
print("Model saved as 'titanic_model.joblib' ✅")

# Verify - load karke test karo
loaded_model = joblib.load('titanic_model.joblib')
test_pred = loaded_model.predict(X_test)
print(f"Loaded model accuracy: {accuracy_score(y_test, test_pred)*100:.2f}%")

Model saved as 'titanic_model.joblib' ✅
Loaded model accuracy: 79.33%
```

Model
Accuracy

79.33%

on test data
(179 samples)

Classification Report Summary

Class	Precision	Recall	F1-Score
Did Not Survive	0.81	0.84	0.83
Survived	0.76	0.73	0.74
Overall Accuracy			79%

Insights & Recommendations

Key Insights

- Females had 74.2% survival rate vs only 18.9% for males
- 1st class passengers survived more (62%) vs 3rd class (24%)
- Children under 10 had higher survival probability
- Passengers paying higher fares were more likely to survive
- Solo travelers had the lowest survival rate
- Cherbourg port passengers had best survival among ports
- Cabin column had 77% missing data was dropped
- No duplicate records found in the dataset

Recommendations

- Try Random Forest or XGBoost for better accuracy
- Apply feature engineering (Title extraction from Name)
- Use cross-validation for more robust evaluation
- Handle class imbalance using SMOTE or class weights
- Tune hyperparameters with GridSearchCV
- Add FamilySize feature (SibSp + Parch + 1)
- Consider ensemble models to improve F1-score for 'Survived' class

Thank You

Titanic Dataset – Exploratory Data Analysis

Data Source: Kaggle | Google Colab | Python | Logistic Regression | Accuracy: 79.33%